Scalar SMU：面向 RISC-V 向量集群原生在线注意力的选择性递推加速

Xiaoteng Wang

摘要—原生在线注意力以增量方式处理分数块，但除首块外，每个分块都需要通过依赖状态的标量递推更新运行最大值与归一化因子。本文研究如何在可编程 RISC-V 向量集群中映射这一递推过程，其伴随的输出更新则具有 D 元素并行性。我们设计了集群内的标量在线合并单元（Scalar Online-Merge Unit，SMU）执行递推，并保留现有 Spatz/RVV 数据通路执行输出更新。硬件通过两条 custom-2 指令暴露给软件：`OMCFG` 写入九个持久化上下文字段并提交初始化；`OMERGE` 对每个后续分数块处理全部已配置的 N 个查询行，并返回完成状态。硬件跟踪交替使用的标量状态缓冲区，软件保留输出向量的管理权。在四个原生工作负载中（块大小为四、种子为一），相对于采用相同 RVV 更新的匹配标量软件递推，MMIO 控制的选择性实现分别在递推窗口、包含控制开销的合并窗口和 Native Core 窗口实现了 8.524–17.784×、4.144–6.275× 和 1.066–1.157× 的加速比。Native Core 不包含 Q/K/V 加载和线性投影，因此并非完整注意力或完整模型的测量。混合精度 SMU 相对于旧版标量实现将独立映射面积降低了 27.613%，但其布局前延迟代理指标更高。

关键词—RISC-V，在线注意力，软硬件协同设计，领域专用加速，向量处理，混合精度计算，Softmax。

I. 引言

Transformer 注意力将矩阵乘法与 Softmax 归一化相结合，是现代序列模型中的基本运算 [1]。在线归一化在分数到达时更新运行统计量 [2]。FlashAttention 将这一思想应用于分块精确注意力，通过感知 IO 的执行方式避免显式构建完整注意力矩阵 [3]；FlashAttention-2 进一步改进 GPU 并行性和工作划分 [4]，FlashAttention-3 则引入异步调度与低精度执行 [5]。在可编程处理器上，在线形式也会在分数块之间形成跨循环迭代的递推依赖。

每次合并包含两类不同的工作。标量递推更新运行最大值和归一化因子，并生成两个用于输出重缩放的权重。这一递推构成依赖链，其运算数量不随输出维度 D 增长。输出更新则将这些权重独立应用于 D 个元素。因此，一次合并将受依赖约束的 $O(1)$ 标量运算与规则的 $O(D)$ 向量运算耦合起来。设计问题在于如何划定专用单元与可编程向量数据通路的边界：统一卸载会重复配置向量资源，而将递推留在标量核上则会保留这一依赖链。

已有研究已经确立了在线归一化方法和专用 Softmax 硬件。Softermax 将在线归一化与低精度算术结合，形成适合硬件的 Softmax [6]；SoftEx 在 RISC-V 集群旁加速 Softmax 和 GELU [7]。VEXP 为 Snitch RISC-V 执行增加近似指数运算，并评估了 Softmax 和 FlashAttention-2 [8]。另有研究复用 Softmax 硬件执行 GELU [9]。RISC-V 平台也覆盖了带加速器接口的标量核 [10]、向量处理器 [11] 和共享内存向量集群 [12]。TOP-V 表明，可以在向量处理器内部专门实现依赖密集型运算，同时保持其周边计算的可编程性 [13]。其可配置排序单元处理 Top-K，而本文研究的单元边界位于原生在线注意力的合并过程内部。

我们在 Spatz RISC-V 向量集群上研究这一划分。集群内的标量在线合并单元（SMU）仅执行依赖状态的递推；现有 RISC-V 向量（RVV）数据通路保留 D 维输出更新。SMU 在递推各阶段采用固定的阶段感知混合精度，并以 FP32 保存内存可见状态。软件通过两条实现相关的 custom-2 指令控制 SMU。`OMCFG` 配置持久化 TCDM 地址、查询行数 N 及所需地址，随后提交上下文；`OMERGE` 对一个后续分数块处理全部已配置的 N 行，并返回完成状态。硬件选择交替使用的标量状态缓冲区，软件负责分块调度、输出缓冲区指针和 RVV 更新。该接口允许 RISC-V 核显式控制加速器，无需增加 SMU 专用向量指令或第二套 D 路数据通路。

在四个原生在线注意力工作负载中，相对于采用相同 RVV 输出更新的标量软件递推，匹配递推窗口的加速比最高达到 17.8×；范围更大的 Native Core 窗口最高仅为 1.16×，且不包含 Q/K/V 加载与线性投影。这并非完整注意力或模型执行结果。独立评估还表明，混合精度 SMU 相对于旧版标量单元降低了独立映射面积，但布局前延迟代理指标更高。指令接口比较显示，测量的递推/控制窗口和配置窗口显著缩短，但 Native Core 没有实质性改善。

本文按主要论点的优先级提出三项贡献：

- 刻画原生在线注意力合并内部的标量/向量不对称性，并评估选择性软硬件划分：Scalar SMU 加速依赖状态的递推，现有 RVV 数据通路保留规则的 D 维输出更新。第 IV-B 节报告这三个测量范围的周期数比值。
- 定义面向这一选择性划分的 RISC-V 编程模型。`OMCFG` 配置九个持久化标量上下文字段并提交初始化；`OMERGE` 对一个后续分数块处理已配置的 N 个查询行并返回状态。硬件适配器跟踪标量 A/B 状态，并将完成响应路由至发起核；软件保留输出缓冲区管理权，使用现有 RVV 指令更新向量。
- 实现集群内的固定阶段感知混合精度 SMU，在 FP32 内存可见接口后采用 FP16、基于查表的定点非线性近似和 Q16.32 归一化累加。其独立映射面积低于旧版标量设计；测量的时序代理指标更高，集群级 PPA 仍不可用。

II. 原生在线注意力与设计动机

A. 原生在线注意力

全文中，N 表示配置的查询行数，B 表示键分块大小；对本文研究的可整除工作负载，$T = N/B$ 表示分块数量；D 表示输出向量长度，t 为分块索引。权重下标中的 *tile* 表示该权重来自分块贡献。撇号表示合并后的状态，帽号表示经过硬件转换的数值。在 Q$I.F$ 格式中，I 为整数幅值位数，F 为小数位数；有符号格式额外包含一个符号位。注意力依据查询与键之间的相似度聚合值向量。传统实现先显式构建分数与归一化概率，再累加值向量。在线

注意力则逐块处理分数, 增量更新归一化状态和输出状态, 从而无需构建完整概率矩阵。这种流式归一化因子是在线 Softmax [2] 和感知 IO 的精确注意力 [3] 的基础。

对于一个查询行, 令 s_i 表示截至当前已处理位置 i 的分数, $\mathbf{v}_i$ 表示对应的值向量。累积状态为 $(m, l, \mathbf{O})$, 并满足式 (1) 的不变量:

$$\begin{aligned} m &= \max_i s_i, \\ l &= \sum_i \exp(s_i - m), \\ \mathbf{O} &= \frac{1}{l} \sum_i \exp(s_i - m)\mathbf{v}_i. \end{aligned} \tag{1}$$

因此, l 是相对于运行最大值 m 的 Softmax 分母, $\mathbf{O} \in \mathbb{R}^D$ 为对应的归一化部分输出。新的分数块提供状态 $(m_t, l_t, \mathbf{O}_t)$。

精确标量合并无需重建先前的概率, 即可合并两组状态:

$$\begin{aligned} m' &= \max(m, m_t), \\ l' &= l \exp(m - m') + l_t \exp(m_t - m'), \\ w_{\text{old}} &= \frac{l \exp(m - m')}{l'}, \qquad w_{\text{tile}} = \frac{l_t \exp(m_t - m')}{l'}. \end{aligned} \tag{2}$$

随后, 向量输出状态按式 (3) 更新:

$$\mathbf{O}' = w_{\text{old}}\mathbf{O} + w_{\text{tile}}\mathbf{O}_t. \tag{3}$$

首个分块直接初始化运行状态。每个后续分块计算递推并更新包含 D 个元素的输出。由于输出始终保持归一化, 因此不需要最终归一化步骤。

B. 执行特征

图 1 将合并分解为依赖状态的标量递推和规则的向量更新。递推更新 m、l 和两个权重; 向量通路将这些权重独立应用于 D 个输出元素。

递推限制了单次合并内部的数据并行性。状态在分块之间传递, 最大值选择、指数重缩放、累加和归一化构成异构依赖链。其标量工作量与 D 无关, 因此增大 D 并不会增加递推的并行通道。

各输出元素 j 呈现相反的模式:

$$\mathbf{O}'[j] = w_{\text{old}}\mathbf{O}[j] + w_{\text{tile}}\mathbf{O}_t[j], \qquad 0 \le j < D, \tag{4}$$

全部 D 个元素共享相同权重, 元素之间不存在依赖。该更新是规则的浮点乘加循环, 适合映射到现有 RISC-V 向量 (RVV) 执行资源。对每个查询行的每个分块, 递推执行固定数量的标量运算, 与 D 无关; 输出更新则对 D 个元素中的每个元素执行两个加权乘积。

因此, 加速器仅针对递推。分数生成、块内状态构建和 D 维输出更新保持可编程, 仅将并行性不随 D 增长的部分移入硬件。对于配置了 N 行的分块合并, 引擎对每行重复执行一次相对于 D 为 $O(1)$ 的递推; RVV 将权重应用于每行的 D 个输出元素。

C. 设计目标

上述执行特征引出三个目标。

第一个目标是选择性专用化: 使用专用硬件执行递推, 并复用 Spatz/RVV 执行规则输出更新。这样既保留了 RVV 可编程性, 又避免重复配置 D 路数据通路。

第二个目标是使递推单元足够轻量, 以适合集群集成。各阶段的数值要求不同, 第 III 节通过阶段感知混合精度处理这些差异。

第三个目标是提供软硬件接口, 使软件能够一次配置工作负载持久化上下文, 随后反复调用合并, 同时由硬件跟踪交替使用的递推状态。

在最终映射中, 集群内的混合精度标量在线合并单元 (SMU) 执行递推, Spatz RISC-V 向量数据通路执行 D 维输出更新, 硬件则跟踪 TCDM 中递推状态的当前 A/B 选择位。

III. 架构与软硬件协同设计

A. 系统概览与软硬件划分

图 2 展示该设计如何集成到 Spatz 向量集群 [12]。Snitch 标量核与 Spatz/RVV 数据通路共享紧耦合数据存储器 (TCDM)。集群内的 SMU 处理第 II 节中的递推, Spatz/RVV 保留规则的 D 维更新。

在线合并 (`OMERGE`) 命令依次经过 Snitch 译码、各核加速器解复用器、集群发射仲裁器和 `OMERGE` 适配器/SMU。仲裁器记录发起核, 响应解复用器将完成响应返回至该核。Spatz/RVV 指令采用直接向量通路。SMU 是独立的 TCDM 请求端, 通过共享存储器读取操作数, 并写入标量状态与权重。默认配置提供 128 KiB、分为 16 个存储体的 TCDM。

该集成不改变 Spatz/RVV 的通道、浮点单元、向量寄存器文件及所支持指令的语义, 也不引入自定义 RVV 指令。SMU 通过 TCDM 交换标量状态和权重, 因此 RVV 例程直接执行式 (3) 和式 (4), 无需 SMU 内部的 D 路数据通路。

查询、键和值 (Q/K/V) 数据以及工作缓冲区均在计时的 Native Core 区域之前准备。该区域不包含内存分配、输入生成、诊断、Q/K/V 加载和线性投影; 包含分数块计算、局部状态构建、首块初始化、后续合并、RVV 更新和最终输出复制。首块将 $(m_t, l_t, \mathbf{O}_t)$ 复制到运行状态。此后, 每条 `OMERGE` 针对一个分数块处理全部已配置的 N 个查询行。对每行, 它读取当前 m, l 和块内 m_t, l_t, 将 m', l' 写入非活动 A/B 缓冲区, 并将两个权重存入 TCDM。全部行完成后, 适配器将状态返回至发起核; 只有成功响应被接收时, 才提交 A/B 选择位, 错误则保持选择位不变。随后 Spatz/RVV 读取权重并更新各输出行, 软件再推进输出指针。第 III-C–III-E 节详细说明配置、调用和选择位语义。

SMU 仅处理标量递推。分数生成、局部状态构建、输出寻址与更新、首块初始化及最终复制仍由软件或 Spatz/RVV 完成。

B. 阶段感知混合精度 *SMU*

SMU 按阶段分配精度, 以适应递推过程中的不同数值范围。该分配是静态的, 如图 3 所示; 输入值不会在运行时选择精度。SMU 计算的是式 (2) 中精确递推的阶段量化近似, 而非精确 FP32 算术。我们用 $\text{RNE}_{16}(\cdot)$、$\text{FP16}(\cdot)$ 和 $\text{Q16.32}(\cdot)$ 分别表示采用舍入到最近偶数方式转换至 FP16、转换至 FP16, 以及转换至 48 位无符号 Q16.32 表示; 帽号标记转换后的值。TCDM 接口以 FP32 传递 m、m_t、l 和 l_t。最大值及其差值采用 FP16 舍入到最近偶数:

$$\begin{aligned} \widehat{m} &= \text{RNE}_{16}(m), \qquad \widehat{m}_t = \text{RNE}_{16}(m_t), \\ \widehat{m'} &= \max(\widehat{m}, \widehat{m}_t), \quad \widehat{\Delta}_{\text{old}} = \widehat{m} - \widehat{m'}, \\ \widehat{\Delta}_{\text{tile}} &= \widehat{m}_t - \widehat{m'}. \end{aligned} \tag{5}$$

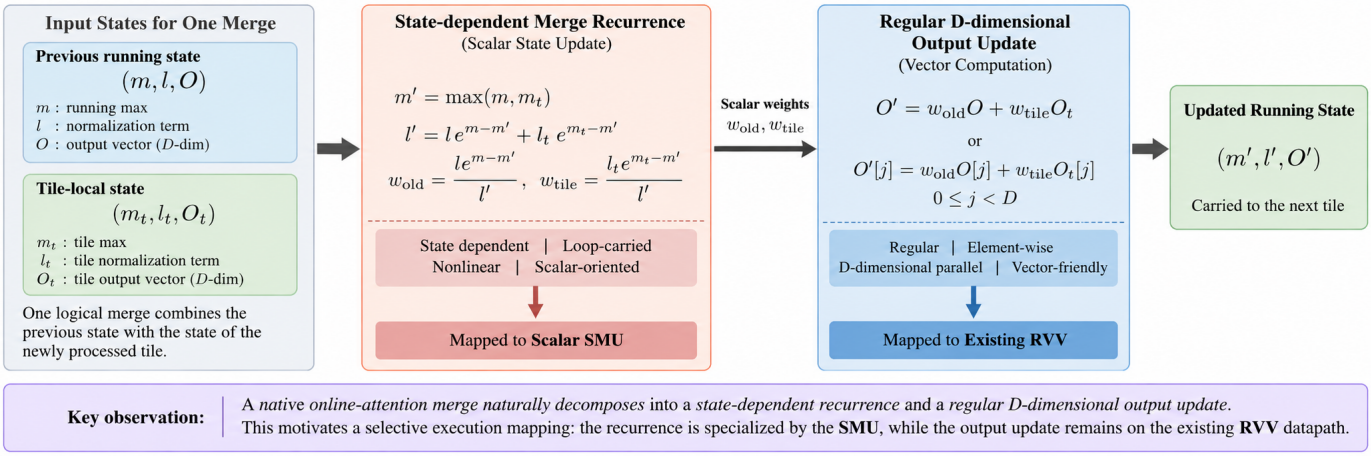


图 1. 原生在线注意力合并的执行分解。依赖状态的递推更新 m 和 l，并生成两个标量重缩放权重。规则的 D 维输出更新仍映射到现有 Spatz 数据通路。

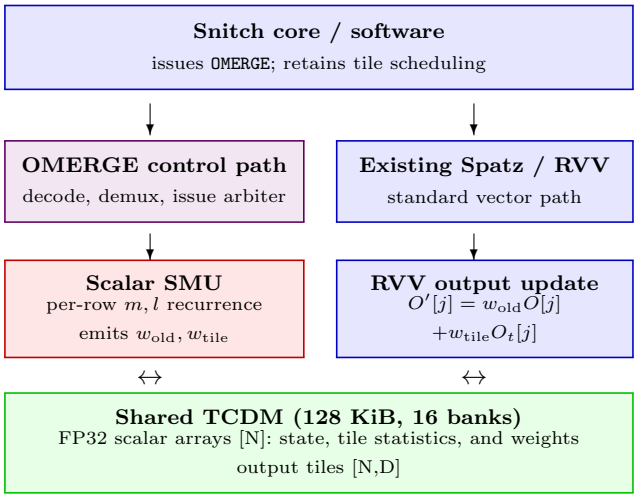


图 2. 系统集成与软硬件划分。Snitch 核通过加速器译码、解复用器和发射仲裁器，将 `OMERGE` 发往 Scalar SMU；适配器向发起核返回完成状态。现有 Spatz/RVV 通路执行规则输出更新。两条通路均通过共享 TCDM 访问逐行标量状态、权重和输出块。

指数（EXP）阶段使用一张含 256 项的有符号 Q1.14 查找表（LUT），其寻址方式为

$$i(\Delta) = \lfloor (\Delta + 8) \cdot 32 \rfloor, \qquad \Delta \in [-8, 0), \tag{6}$$

表项编码对应区间中心，不进行插值。胜出贡献满足 $\Delta \ge 0$，绕过 LUT 并直接使用精确单位值，因此转换后的最大值相等时，两条通路均输出单位值。只有较小贡献对应的差值访问 LUT；EXP 通路仅在 $|\Delta| > 8$ 时对其进行饱和处理，而根据式 (6)，$|\Delta| = 8$ 时访问 LUT[0]。将这些缩放因子记为 $\hat{s}_{\text{old}}$ 和 $\hat{s}_{\text{tile}}$。

SMU 将 FP32 的 l 和 l_t 转换为无符号 Q16.32 表示，分别记作 $\hat{l}$ 和 $\hat{l}_t$，并计算

$$\hat{l}' = \hat{l}\,\hat{s}_{\text{old}} + \hat{l}_t\,\hat{s}_{\text{tile}} \tag{7}$$

式 (7) 中的和保持为无符号 Q16.32 表示。在计算倒数前，分母经过 FP16 RNE 转换边界。对于经过 FP16 舍入并规范化的分母 $\hat{x} = 2^{e-127}(1+\mu)$，其中 μ 是规范化尾数的小数部分，倒数近似为

$$\frac{1}{\hat{x}} \approx 2^{\,127-e}\Big[L_j + (L_{j+1} - L_j) f_j\Big], \qquad j = \lfloor 256\mu \rfloor, \quad f_j = 256\mu - j. \tag{8}$$

其中 L_j 是 $1/(1 + j/256)$ 的无符号 Q1.15 编码，f_j 由尾数低 15 位承载。式 (8) 给出的倒数用于缩放每个分子。每个缩放后的分子也经过 FP16 RNE 转换边界，并在计算权重之前回到 Q16.32。一个共享的 48×16 乘法器依次生成旧状态权重和分块权重。倒数 LUT 通路由构建宏 `ONLINE_MERGE_MIXED_RECIPROCAL` 选择；其来源信息随评估结果一并报告。

所支持的数值约定要求 FP16 转换后的分母为正有限值，且定点中间结果不溢出。Q16.32 存储并未消除 FP16 归一化边界。定向模型测试向量覆盖零分子、次正规数和最大有限 FP16 分母；不支持零除以零。任意非有限输入以及溢出恢复不属于已验证工作负载的约定范围。EXP 截断和 FP16 舍入仍是总递推误差的来源。

引擎将 $\widehat{m'}$ 和 $\widehat{l'}$ 编码为 FP32，写回 TCDM。该编码仅恢复内存侧的表示类型，不能恢复已丢弃的精度：最大值来自 FP16 舍入，归一化项来自 Q16.32。权重通过式 (5) 所述转换成为 $w = \mathrm{FP32}(\mathrm{FP16}(\widehat{w}))$，随后由保持不变的 FP32 Spatz/RVV 更新消费。因此，FP32 承载内存可见状态，Q16.32 承载归一化范围，FP16 与定点查表硬件处理有界非线性阶段。所有分配均在设计时固定。

C. *RISC-V* 指令接口

编程模型将持久化上下文配置与反复执行分开。两种操作都采用 RISC-V 为实现相关扩展保留的 custom-2 操作码空间 [14]，并进入 Spatz 集群现有的 Snitch 加速器请求通路 [12]。表 I 列出其编码和架构效果，表 II 给出精确译码模式。两条指令均不改变 RVV 编码或受支持向量指令的语义。

表 II 给出精确译码掩码、匹配值和机器字表达式。固定功能位将反复执行命令与配置命名空间分开，同时将两种操作均保留在 custom-2 中。

`OMERGE` 是标量递推命令，而非向量操作。其 `rs1` 和 `rs2` 字段均指定 `x0`。每条命令对一个分数块中的全部已配置 N

表 I
CUSTOM-2 指令格式及架构效果。

指令	编码字段	寄存器与立即数字段	架构效果
`OMERGE`	R 型；操作码 `0x5B`；funct7 `0000011`；funct3 `000`	`rs1=rs2=x0`；`rd` 为状态目标寄存器	针对一个分数块处理全部已配置的 N 行，通过 TCDM 读写每行的标量递推状态，存储两个权重，并返回完成状态。
`OMCFG`	类 I 型；操作码 `0x5B`；funct3 `001`；`rs1` 提供数值；`rd=x0`	`imm12=cfg_id`	写入一个持久化上下文字段，或发出独立的 INIT 提交操作。不启动递推，也不返回寄存器结果。

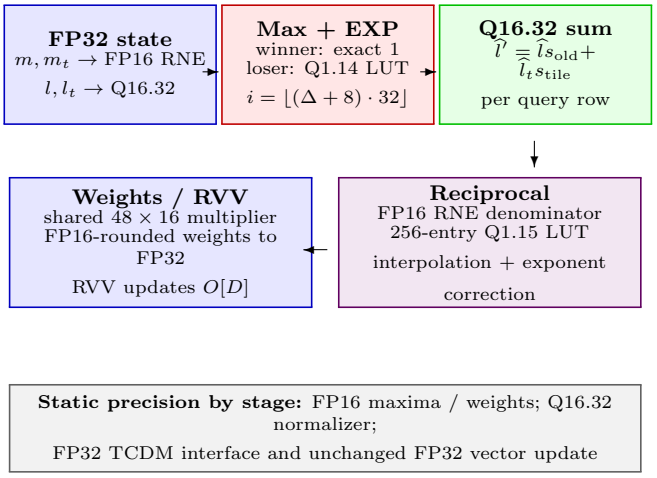


图 3. 阶段感知混合精度 Scalar SMU 数据通路。图中展示逐行 FP32 状态、FP16-RNE 最大值与倒数转换边界、Q16.32 归一化、直接区间 EXP 查表、插值倒数修正，以及供现有 RVV 输出更新使用的 FP16 转 FP32 合并权重。

表 II
两条 CUSTOM-2 命令的精确译码模式。当指令字与掩码按位与的结果等于匹配值时，适配器识别该指令。

命令	掩码、匹配值与机器字
`OMERGE`	掩码 `0xFFFFF07F`, 匹配值 `0x0600005B`, 机器字 `0x0600005B | (rd << 7)`; `instruction[11:7]` 编码 `rd`, 即完成状态目标寄存器，而 `rs1=rs2=x0`.
`OMCFG`	掩码 `0x00007FFF`, 匹配值 `0x0000105B`, 机器字 `(cfg_id << 20) | (rs1 << 15) |0x0000105B`; 12 位 `cfg_id` 占据位 [31:20]，`rs1` 在位 [19:15] 指定字段值的源寄存器，且 `rd=x0`.

行执行标量递推，通过 TCDM 读取 m、l、m_t 和 l_t，并写入 m'、l' 及两个重缩放权重。D 维输出数据不经过指令接口。适配器将引擎固定为 Mixed Scalar 模式，与 MMIO 控制的 SMU 通路采用相同算术；现有 RVV 例程执行输出更新。

`rd` 字段指定状态目标寄存器。软件必须使用非零目标寄存器才能检查返回状态；`rd=x0` 会丢弃寄存器写入。发射时，Snitch 在其加速器记分牌中预留目标寄存器，并通过各核加速器解复用器和集群发射仲裁器转发指令与请求标识。适配器只允许一条未完成命令，依次经历 `IDLE`、`START`、`WAIT` 和 `RESP` 状态。在 `WAIT` 中，它等待 SMU 报告 DONE 或 ERROR。在 `RESP` 中，响应通路向发起核返回请求标识和状态；接收该响应后写入 `rd`，并清除相应的待完成记分牌条目。状态为零表示成功，非零表示错误。因此，只有核接收加速器响应后，软件才能认为该指令已完成。

反复执行的计时比较采用完整通路：发射、SMU 执行与忙碌时间、响应提交以及状态消费。这是包含全部相关开销的递推/控制窗口，并非仅包含派发的延迟。单独的 `OMERGE` 接口比较保持 SMU 算术和 RVV 更新相同，仅改变每个非首块递推的调用方式。

D. 通过 *`OMCFG`* 实现运行时可配置上下文

地址与工作负载参数需要在一系列分块合并之间保持。如果围绕每次递推重写这些参数，就会将配置流量重新引入稳态通路。`OMCFG` 一次写入工作负载持久化上下文；此后每个分数块通过一次 `OMERGE` 调用处理全部已配置查询行。

图 4 将四类管理职责放在执行时间线上：`OMCFG` 建立持久化上下文，`OMERGE` 推进标量状态，硬件在成功完成后翻转选择位，软件更新输出缓冲区指针。

`OMCFG` 使用与 `OMERGE` 相同的 custom-2 操作码，并选择 funct3 `001`。类 I 型编码将 12 位配置标识放入 `imm12`，由 `rs1` 提供数值；`rd` 固定为 `x0`。该操作不创建目标寄存器记分牌条目，也不返回写回载荷。只有适配器和 SMU 引擎均空闲时，适配器才接受配置命令。立即数选择字段，并不承载字段值。

标识 `0x000`–`0x008` 写入九个必需字段，指定两组运行状态缓冲区、块内 m_t/l_t、两个权重数组的物理 TCDM 基址，以及查询行数 N。标识 `0x009` 选择独立的 CTRL/INIT 操作；其数值的位 0 提交完整上下文并复位选择位。表 III 列出该映射及其边界。每个配置地址都是逐行 FP32 数组的基址；`scalar_addr(base,row)=base+(row<<2)` 将行 r 放在字节偏移 $4r$ 处。

九个必需字段与旧版 MMIO 接口共享规范的生成寄存器。`OMCFG` 写入更新与 MMIO 相同的寄存器，因此 MMIO 读取可观察到写入值。适配器使用 `9'h1FF` 掩码跟踪写入，并将上下文标记为脏。只有九个必需位全部存在时，位 0 置位的 INIT 才能提交；成功后置位 `cfg_valid` 并将选择位复位为零。位 0 清零的 INIT 不提交。重写任何必需字段后，必须在下一次合并之前再次执行完整 INIT。

这是对刻意限定的上下文提供运行时可配置性，而非通用 SMU 控制 ISA。版本 1 不提供 D、精度或模式、块大小、输出向量地址、步长或选择位的配置标识。适配器固定采用 Mixed Scalar 模式，并为标量模式中未使用的字段提供验证常量，包括 $D = 1$ 和零步长。工作负载维度与输出寻址仍是软件/RVV 参数。保留标识不执行寄存器写入，并使就绪状态失效；软件必须在下一条 `OMERGE` 前执行完整 INIT。因此，接口评估将一次性配置、反复调用及范围更大的 Native Core 窗口分开。

E. 调用序列与状态管理权

配置地址指定两组物理运行状态缓冲区。一个硬件选择位标识哪组提供当前标量状态，哪组接收下一状态，从而避

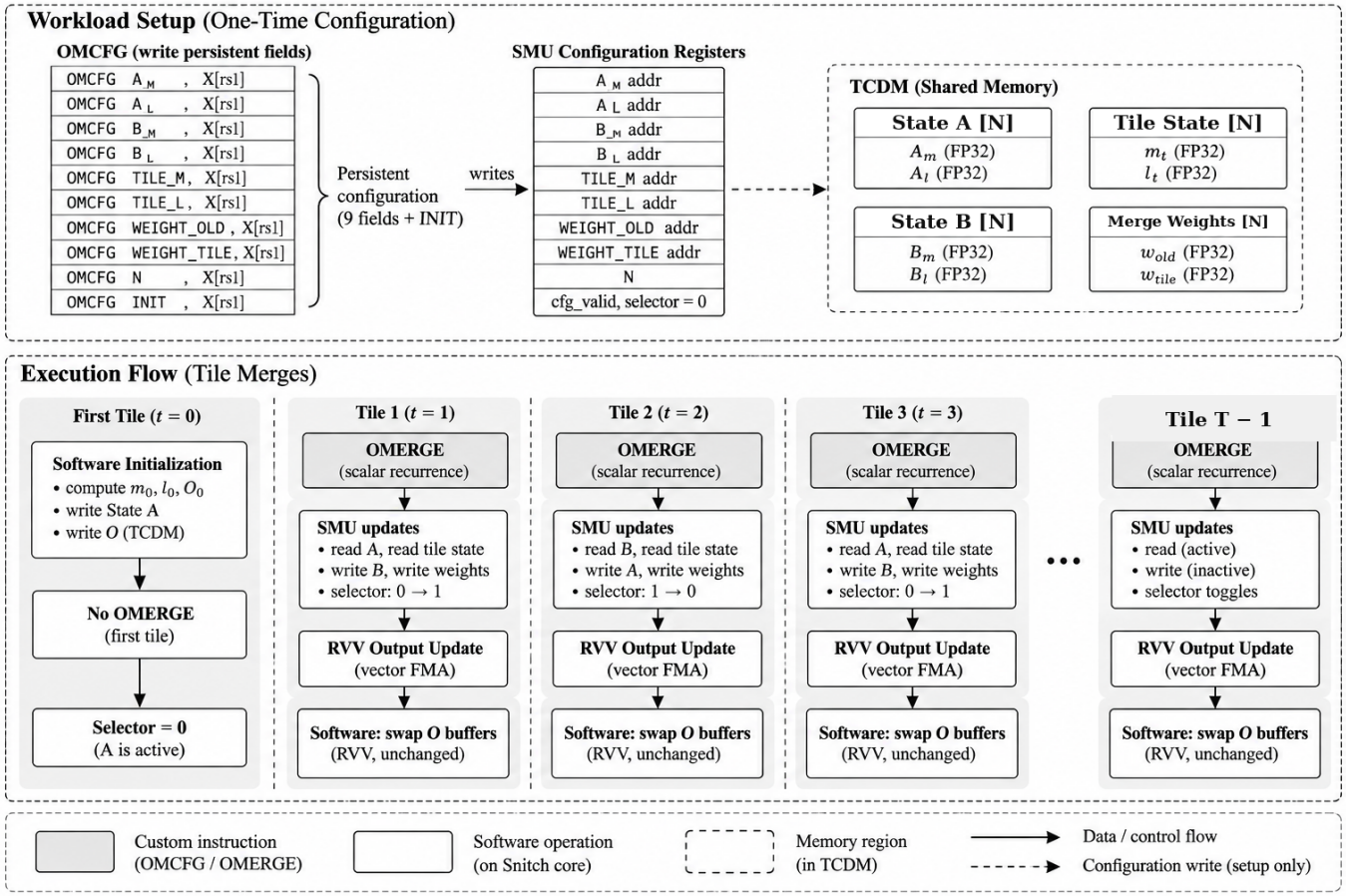


图 4. OMCFG/OMERGE 编程模型。一次性的 OMCFG 写入建立持久化标量上下文并复位选择位。每个后续分数块使用一条 OMERGE 命令处理已配置的 N 个查询行，随后执行不变的 RVV 输出更新。当块大小 $B = 4$ 时，包含 N 行的工作负载共有 $T = N/B$ 个分块，最后一个分块为 $T - 1$。虚线配置通路写入的是上下文地址，而非 TCDM 数据。硬件仅翻转 A/B 标量状态选择位；软件交换输出缓冲区。选择位在发起核接收成功响应后才改变。

表 III
OMCFG 配置标识映射。标识 0–8 为必需上下文字段；标识 9 是独立的 INIT 控制操作。

cfg_id	字段	数值或地址	用途与写入语义
0x000	A_M	TCDM 地址	状态缓冲区 A 中逐行运行最大值数组
0x001	A_L	TCDM 地址	状态缓冲区 A 中逐行运行归一化因子数组
0x002	B_M	TCDM 地址	状态缓冲区 B 中逐行运行最大值数组
0x003	B_L	TCDM 地址	状态缓冲区 B 中逐行运行归一化因子数组
0x004	TILE_M	TCDM 地址	逐行块内最大值数组 m_t
0x005	TILE_L	TCDM 地址	逐行块内归一化因子数组 l_t
0x006	WEIGHT_OLD	TCDM 地址	逐行数组基址：w_{old}
0x007	WEIGHT_TILE	TCDM 地址	逐行数组基址：w_{tile}
0x008	N	查询行数	每次 OMERGE 调用处理的行数
0x009	CTRL/INIT	位 0 = 1	提交全部必需字段并复位 A/B 选择位
0x00A--0xFFF	保留	不写寄存器	不支持的标识使就绪状态失效，直到执行完整 INIT

免软件在每次合并时更新地址。图 4 展示其生命周期：软件初始化首块及输出状态, 写入九个必需字段, 提交 INIT, 随后对每个后续分块发出一条 OMERGE。成功响应之后，现有 RVV 例程更新输出，软件交换其输出缓冲区指针。

选择位作用于 TCDM 中的两组缓冲区 $A = (A_m, A_l)$ 和 $B = (B_m, B_l)$；对每个查询行 $0 \le r < N$，每组均保存一个 FP32 最大值和一个 FP32 归一化因子。

选择位为零时，选择 A 作为源、B 作为目标；选择位为一时，选择 B 作为源、A 作为目标。选择位改变提供给 SMU 的物理 m/l 地址。缓冲区仍位于 TCDM 中，并保持原有内容。

软件将首块作为初始化处理：把初始运行标量状态写入 A，并在软件中初始化输出缓冲区，因此首块不执行 OMERGE。完整 INIT 使选择位保持为零。第一次后续合并读取 A、写入 B，此后的合并交替改变方向。

只有发起核接收无错误的加速器响应时，适配器才提交选择位转换。指令发射和 SMU 启动均不改变选择位。不完整或脏上下文返回非零状态，且不置位引擎启动信号。SMU 错误也返回非零状态。两种情况均保持选择位原值，因此失败命令不会改变下一次成功合并的状态方向。

选择位只管理逐行标量 m/l 状态。软件继续维护两个 D 维输出缓冲区指针，不变的 RVV 例程在每次成功递推后

执行输出更新。分块地址与输出指针也由软件负责。本文评估的约定采用一个控制核和一个活动上下文。请求归属路由不提供各核上下文隔离。并发配置写入者、中断、取消和故障恢复均不属于已验证范围。软件在 RVV 更新前消费成功完成状态；两个锚点测试确立的是输出等价性，而不是并发执行下内存顺序的一般证明。

IV. 评估

A. 实验设置与测量范围

性能结果来自 Spatz RISC-V 向量集群的周期级仿真。Verilated 模型采用 Verilator 5.034 和默认集群配置，其中 TCDM 容量为 128 KiB，分为 16 个存储体。周期计数器明确界定各测量阶段，每组匹配比较均使用同一冻结的集群配置和软件工具链。

N8/D32 和 N16/D64 是两个锚点工作负载。N24/D64 与 N16/D128 用于检验序列长度或输出维度改变后，已观察行为是否仍然成立。所有原生注意力案例均采用块大小 $B = 4$ 和种子一。首块直接初始化运行状态，不需要合并。因此，每个查询行有 $T - 1$ 次后续分块合并：N8/D32 为一次，N16/D64 为三次，N24/D64 为五次，N16/D128 为三次。每个后续分数块发出一条 `OMERGE`，处理全部已配置的 N 行。

该设计仅加速原生在线注意力的一部分，因此我们报告三个计时范围。B2R 软件的递推窗口涵盖生成 m'、l'、w_{old} 和 w_{tile} 的依赖状态标量更新。对于匹配的 MMIO 控制 SMU 通路，所报告递推窗口还包括每次合并的寄存器配置、清除/启动、BUSY/DONE 轮询和 SMU 执行。因此，它测量完整的递推调用，包含控制和引擎忙碌时间，而非纯算术延迟。合并窗口进一步包含对应的 RVV 输出更新及合并侧编排。*Native Core* 窗口再包含分数块计算、块内状态构建、首块处理、全部后续合并及最终输出处理。内存分配、输入生成、诊断、Q/K/V 加载和线性投影均不属于这三个窗口。

各比较构成独立的匹配实验组。算术卸载组将标量软件递推加 RVV（B2R）与 MMIO 控制的 Mixed Scalar SMU 加相同 RVV 更新进行比较。反复调用接口组保持 SMU 算术与 RVV 相同，比较逐次合并 MMIO 控制与 `OMERGE`。配置组保持 `OMERGE` 相同，比较一次性 MMIO 配置与 `OMCFG`。Full SMU 仅出现在独立的历史消融实验中。

每组使用其各自匹配的基线和冻结实现。不同实验组之间的绝对周期数不能构成优化演进轨迹，且没有 B2R 与最终 `OMCFG`/`OMERGE` 接口之间的匹配比较。B2R 实现相同的数学递推并共享 RVV 例程，但其内部数值格式与 SMU 不同。因此，递推比值同时包含硬件专用化与数值实现差异，本文未将二者分离。B2R 不是优化软件库基线。所有原生案例仅采用一个种子，接口实验组对每个通路和形状仅有一次正式运行。这些结果确立的是确定性原型行为，而非模型准确率、多头扩展性或统计普适性。加速比均为周期数比值，不假设已经达到某一工作频率。

B. 原生在线注意力性能

原生比较保持输出更新相同，以测量递推卸载的效果。B2R 是匹配软件基线：它在标量软件中执行递推，并共享 SMU 通路的 RVV 输出例程。图 5 汇总三个计时范围。

在 N8/D32 中，递推从 12,368 降至 1,451 周期，加速比为 8.524×。合并窗口从 13,204 降至 2,318 周期，加速比为 5.696×。Native Core 从 85,262 降至 73,706 周期，加速比为 1.157×，周期数降低 13.554%。

在 N16/D64 中，递推从 68,994 降至 4,926 周期，加速比为 14.006×。合并窗口从 77,037 降至 13,060 周期，加速比为 5.899×。Native Core 从 571,374 降至 507,918 周期，加速比为 1.125×，周期数降低 11.106%。

N24/D64 在递推、合并和 Native Core 范围内分别达到 17.784×、6.275× 和 1.140× 加速比。N16/D128 在相同三个范围内分别达到 13.737×、4.144× 和 1.066×。四个工作负载的测量范围分别为：递推 8.524–17.784×，合并 4.144–6.275×，NativeCore 1.066–1.157×。

测量范围越大，该设计未改变的工作所占比例就越高：分数构建、块内状态生成和规则向量输出更新均保留在可编程通路。因此，Native Core 的改善小于递推，递推结果也不能表述为注意力级或模型级加速。

C. 数值保真度与独立 *SMU* 成本

阶段感知混合精度 SMU 与匹配的 B2R 软件递推高度一致，同时降低独立映射面积。在四个原生注意力工作负载中，最终 Mixed Scalar SMU 的余弦相似度为 0.999970724 至 0.999984751。这些案例中没有非有限输出。

表 IV(a) 将原生案例与 B2R 软件输出比较；指标由存储的 FP32 结果以 float64 计算。FP32 是两条通路的存储类型，并非独立参考真值。单独的数值压力扫描中，最低余弦相似度降至 0.999873853，因此不能声称所有情况均高于 0.9999。穷举全部 31,743 个正有限 FP16 分母得到的最大倒数相对误差为 6.6966×10^{-5}。独立的主机模型扫描覆盖随机分数与单头 Q/K/V 案例，长度为 128、256、512 和 1024，维度为 64 和 128（共 16 种组合），块大小为 32，基础种子为 20260823。它不同于块大小为四的 RTL 原生性能实验。所选倒数扫描中，替换实现相对于除法实现的最低余弦相似度为 0.999999885，而优化实现相对于 FP32 的最低值为 0.999873853。这些模型结果并未确立对应长度下的 RTL 性能。

阶段感知精度的成本通过匹配的独立 Yosys/ABC 综合测量，采用 Nangate45 典型工艺库。将旧版算术替换为混合精度并保留除法后，映射面积从 72,399.082 降至 52,000.074 单位，降低 28.176%。最终倒数实现占用 52,407.586 单位，比旧版低 27.613%，比 Mixed+Division 高 0.784%。

更低面积并不意味着更优时序。最长映射寄存器间延迟代理指标从旧版的 12.518 ns 上升至 Mixed+Division 的 14.620 ns 和 Mixed+Reciprocal 的 15.540 ns。这些数值仅是布局前时序代理指标，且没有设计达到 1.5 ns 目标。集群综合在 techmap 阶段耗尽资源，因此集成面积、频率和功耗仍未测量；这里的面积比较仅针对独立单元。

D. 软硬件划分消融

最终架构将 D 维输出更新保留在 RVV 上，SMU 内不设置专用向量数据通路。我们通过独立冻结的 Full SMU/A2 实验考察这一选择。该消融早于最终 `OMERGE`/`OMCFG` 实现，因此其绝对面积和周期值仅在各自匹配的历史实现阶段内使用。

在该实验中，选择性设计将标量递推卸载与 RVV 输出更新结合；Full SMU 进一步将随维度变化的输出更新移入加速器。独立面积从 77,103.292 增至 114,713.298 单位，

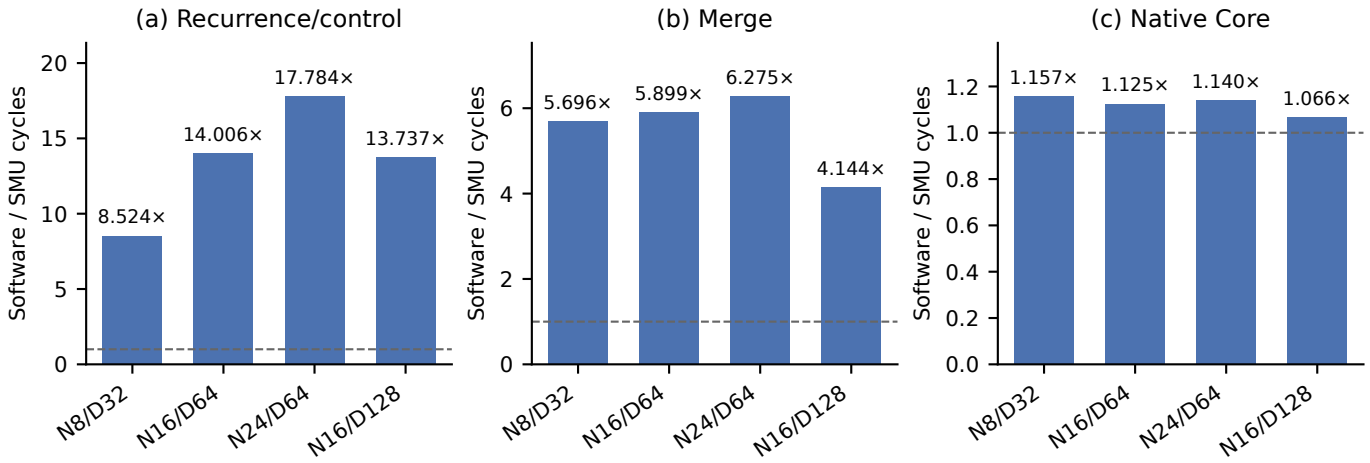


图 5. 原生在线注意力在递推、合并和 Native Core 测量范围内的加速比。MMIO 控制的 SMU 递推窗口包括每次合并的配置、清除/启动、轮询和 SMU 执行。比值采用匹配软件与 SMU 周期数，块大小为四，种子为一。每个柱形对应一个确定性案例；未估计不确定性区间。各子图的刻度不同。

表 IV
与 B2R 软件递推的数值一致性及独立 SMU 成本。面积来自面积映射流程，延迟代理指标来自对相同设计采用独立时序约束的映射流程；两列并非描述同一网表。

(a) 与 *B2R* 软件输出的一致性

形状	MAE	余弦相似度
N8/D32	2.793084×10^{-4}	0.999984541
N16/D64	4.665494×10^{-4}	0.999970724
N24/D64	3.736880×10^{-4}	0.999980879
N16/D128	5.631592×10^{-4}	0.999984751

(b) 独立综合

设计	映射面积	相对旧版的面积降幅	延迟代理指标
旧版标量设计	72,399.082	不适用	12.518 ns
Mixed+Division	52,000.074	28.176%	14.620 ns
Mixed+Reciprocal	52,407.586	27.613%	15.540 ns

表 V
选择性卸载与完整卸载的历史消融。每行内部使用相同周期窗口和实现阶段，与主要原生注意力实验分开。

形状	选择性设计	Full SMU	完整/选择性
N12/D64	4,128	7,704	1.866×
N32/D128	12,866	34,728	2.699×
N40/D128	15,733	43,206	2.746×

增加 48.779%。历史映射延迟代理指标也从 12.34285 增至 13.20260 ns。

这些是历史工作负载的测量，并非完整 BERT、Mistral 或 Qwen 模型运行时间。三个匹配案例中 Full SMU 均更慢。这些数值不纳入最终原生注意力结果。它们与面积比较共同支持选择性划分：规则的 D 维工作保留在 RVV 数据通路，而非在 SMU 内重复实现。

E. RISC-V 指令接口评估

ISA 接口评估分两阶段：先评估每次合并的反复调用，再在保持 `OMERGE` 相同的条件下评估一次性工作负载配置。

A1-MMIO 与 A1-ISA/`OMERGE` 使用相同的 Mixed Scalar SMU 算术、RVV 输出例程、工作负载、块大小和数值语义，仅每个非首块递推的触发方式不同。表 VI(a) 以周期为单位报告两个锚点的匹配窗口。

包含相关开销的递推/控制窗口覆盖发射与控制活动、SMU 执行及忙碌时间、响应提交和状态消费，因此其缩短不只是派发延迟的下降。窗口的大部分时间是 SMU 忙碌时间，两条通路承担相同或近似相同的成本。扣除该时间后，可分离递推周围的控制活动：N8/D32 从 1,261 降至 40 周期，N16/D64 从 3,577 降至 81 周期。扩大范围会加入接口未改变的工作，因此合并范围的降幅减小至 41.695% 和 21.954%；当一次性 ISA 配置纳入 Native Core 窗口时，降幅进一步减小至 0.841% 和 0.448%。

在保持 `OMERGE` 相同的条件下，我们比较 MMIO-config+`OMERGE` 与 `OMCFG`-config+`OMERGE`。两个变体执行相同的基于 `OMERGE` 的 Native Core，仅在核心执行前传送九个配置字段和一次 INIT 操作的方式不同。N8/D32 的配置从 991 降至 659 周期，降低 33.50%；N16/D64 从 986 降至 632 周期，降低 35.90%。N8/D32 的 Native Core 执行完全相同，N16/D64 仅相差 12 周期；对应的包含配置的总周期数分别降低 0.451% 和 0.067%。表 VI 汇总两种比较。

两种接口比较均保持功能行为不变。`OMERGE` 与 MMIO 控制通路在两个锚点工作负载中生成逐位相同的最终输出。N8/D32 发出一条 `OMERGE`，N16/D64 发出三条，与预期非首块合并次数及选择位转换次数一致；每条命令处理全部已配置查询行。N8/D32 的最终输出哈希为 `0x9c7cdedc`，N16/D64 为 `0xfd0d2cff`。匹配输出的成对 MAE 和稳定相对误差均为零，余弦相似度为一，且没有非

表 VI
ISA 接口评估。在 (A) 中，两条通路采用相同 Mixed Scalar SMU 算术与 RVV 例程，因此递推和合并窗口中的 SMU 忙碌周期相互抵消或近似抵消（N16/D64 相差 4 周期）。控制行扣除共同忙碌周期，仅保留启动、轮询与提交活动。

(a) OMERGE 逐次合并效果（周期）

形状	窗口	MMIO	OMERGE	降幅
N8/D32	控制（不含忙碌）	1,261	40	96.828%
	递推/控制	2,069	848	59.014%
	合并	2,926	1,706	41.695%
	含配置的 Core	74,392	73,766	0.841%
N16/D64	控制（不含忙碌）	3,577	81	97.736%
	递推/控制	8,429	4,929	41.523%
	合并	16,548	12,915	21.954%
	含配置的 Core	510,991	508,702	0.448%

(b) OMCFG 配置效果

形状	MMIO 配置	OMCFG 配置	配置降幅	Native Core Δ
N8/D32	991	659	33.50%	0
N16/D64	986	632	35.90%	+12

有限值。反复调用接口实验未归档中间 m/l 和权重数组，因此不声称中间结果逐位一致。

每个二进制文件均包含两种配置例程。驻留在数据段中的 `volatile` 通路选择变量在两次构建中具有不同初值，并在运行时选择 MMIO 或 OMCFG。因此，同一工作负载中 MMIO-config 与 `OMCFG`-config 二进制的 `.text` 逐字节相同，并生成逐字一致的最终输出。INIT 后，它们还建立相同的持久化配置状态，包括全部九个字段和为零的选择位。因此，`OMCFG` 降低了测量的一次性配置成本，但没有测得系统性的稳态 NativeCore 收益。

F. 显式 P 负对照

最后一个实验检验 SMU 能否在显式概率矩阵数据流中作为逐行 Softmax 的通用替代。周边量化注意力通路保持不变；仅将显式 P 软件 Softmax 后端替换为 Mixed SMU 后端。

对于 N8/D32，Mixed SMU 将显式 P Softmax 窗口从 12,502 增至 14,812 周期，比软件慢 18.477%。对于 N16/D64，窗口从 48,595 变为 48,534 周期，改善 0.126%。两个案例中，采用 SMU 的更大范围注意力窗口均略慢。

五阶段 Attention Total 在 N8/D32 中从 839,111 增至 841,597 周期，增加 0.296%；在 N16/D64 中从 6,767,387 增至 6,768,406 周期，增加 0.015%。完整 Mixed Softmax 窗口包含 P 初始化、SMU 启动与轮询、RVV 概率更新以及交换/清除编排。五阶段 Attention Total 是算术项的求和，并非来自独立计时器。

这些结果与第 IV-B 节的原生在线注意力结果形成对照。作为通用逐行 Softmax 替代，SMU 在这两个案例中没有提供一致收益。原生路径的正面结果和这一负对照共同界定了实现的已测适用范围；它们并不能证明其他 Softmax 实现无法获益。

V. 讨论

A. 收益取决于所选执行边界

测量收益在递推边界最大，随着窗口纳入保留在可编程通路上的工作而减小。分数块计算与规则的 D 元素输出更新仍在标量加速器之外；因此，Native Core 结果反映该划分对测量内核的影响，不包含 Q/K/V 加载和线性投影。

原生与显式 P 的测量共同支持一个与边界相关的结论：SMU 针对原生在线注意力的持久化标量递推，而不是普遍更快的 Softmax 后端。

B. 指令接口向软件暴露这一划分

软件通过 `OMCFG` 写入九个受支持上下文字段并提交就绪状态，随后对每个后续分数块调用 `OMERGE`。硬件处理全部已配置查询行，并跟踪交替使用的标量状态选择位；软件保留分块调度、输出缓冲区指针和 RVV 执行。核的加速器通路暴露这一有状态操作，无需自定义向量指令或重复配置输出数据通路。

测量不支持将指令接口视为主要加速收益的来源。`OMERGE` 在两个锚点将包含相关开销的递推/控制窗口缩短 41.5–59.0%，但该窗口包含 SMU 忙碌时间和响应处理，并非仅包含派发的延迟。`OMCFG` 将一次性配置缩短 33.5–35.9%；包含配置的总周期数分别变化 0.451% 和 0.067%。这些控制平面结果补充了选择性数据流结果，并未确立任一指令单独带来的实质性稳态 Native Core 收益。

该接口的粒度不同于 VEXP 标量与打包 SIMD 指数指令等既有 RISC-V 非线性扩展。本文命令暴露的是在线合并递推及其持久化状态，而非通用指数运算原语。这一区别限定了本文的定位；它并不意味着在线归一化、Softmax 硬件或 RISC-V 非线性加速本身是新概念。

C. 数值与评估局限

匹配的集群综合在 techmap 阶段耗尽资源，因此未测量集成面积、频率和功耗。B2R 是共享 RVV 例程的匹配递推软件，而非成熟的优化软件库基线。

VI. 与既有工作的比较

表 VII 将本文与代表性的 RISC-V 和 AI 加速方案进行定位比较；下文各小节给出每行的详细背景。

A. *Softmax* 与非线性加速

当矩阵乘法得到充分优化后，指数运算、Softmax 和 GELU 等非线性算子会成为 Transformer 工作负载的瓶颈。已有研究在多个粒度上解决这一问题，从算术 ISA 扩展和低成本可重配置非线性核 [18], [19]，到专用非线性加速器。

VEXP 在 Snitch RISC-V 核的浮点单元内部加入近似 BFloat16 指数单元，通过标量 `FEXP` 和打包 SIMD `VFEXP` 指令暴露 [8]。其 Softmax 内核将该指数运算与现有 Snitch 执行机制结合，报告最高 162.7× 的 Softmax 延迟降低，以及 GPT-2 配置下 8.2× 的 FlashAttention-2 加速。VEXP 是细粒度方案：将代价高昂的非线性原语加入可编程数据通路。

SoftEx 采用另一种路线，在专用 BFloat16 硬件中加速整个非线性算子 [7]。其数据通路覆盖近似指数运算、累加、求逆和归一化，并位于 RISC-V 集群中矩阵引擎旁。扩展设计还支持 GELU，并报告最高 10.8× 的 Softmax 加速，展示了在已加速矩阵通路旁增加非线性硬件的收益。

本文比较聚焦另一种接口：持久化归一化状态与逐次合并控制。在第 IV-F 节的显式 P 负对照中，SMU 并未一致改善逐行 Softmax。测量收益出现在原生通路中，其中标量状态在在线合并之间持续保存，而 D 维输出更新保留在 RVV 上。

表 VII
代表性 RISC-V 与 AI 加速方案的定位。所报告结果保留各原工作的测量范围，不能跨行直接比较。

工作	硬件通路及其与向量执行的关系	报告结果
VEXP [8]	Snitch FPU 通过自定义标量/打包指令 `FEXP`/`VFEXP` 执行 BF16 指数运算	162.7× Softmax 延迟降低; 8.2×FlashAttention-2 加速比
SoftEx [7]	RISC-V 集群 HWPE 通过控制与流式接口加速 Softmax/GELU	10.8× Softmax 加速比; 1.58× 端到端 ViT 吞吐量提升
TOP-V [13]	带自定义 `VTOPK` 排序指令的 Ara 向量处理器	5.24× 相对基于 RVV 排序的加速比
RedMulE [15]	共享集群存储器的集群内 HWPE 风格 GEMM 引擎	15× GEMM; 最高 62× GEMM-Op 相对软件的加速比
Gemmini [16]	通过 RoCC 连接的脉动阵列及完整软件栈	全栈 DNN 评估；相对 CPU 基线提升多个数量级
Deeploy [17]	编译器将 SLM 执行映射至 RV32 核、ML 扩展和 NPU	340 token/s，能耗为 490 μJ/token (TinyStories SLM)
本文	集群内 Scalar SMU 通过 `OMCFG`/`OMERGE` 专门处理递推；Spatz/RVV 保留输出更新	最高 17.784× 递推加速比、6.275× 合并加速比和 1.157× Native Core 加速比（MMIO 控制 SMU，相对 B2R 软件基线）

B. RISC-V 领域专用加速

RISC-V 的可扩展性使领域专用硬件能够保持在软件控制之下，无需独立编程模型。现有系统涵盖紧密集成的功能单元扩展 [10], [8]、集群内协处理器 [15], [7] 以及向量处理器扩展 [11], [13], [12]。

Spatz 提供本文使用的可编程向量基础：它是面向紧耦合共享内存集群的紧凑 RISC-V 向量处理器，具有规则向量执行能力和相对较小的向量寄存器文件 [12]。本文设计保留该向量数据通路执行规则输出更新，不改变 RVV 执行语义。

TOP-V 是选择性专用化的另一个例子 [13]。混合专家路由需要分支密集、依赖密集的 Top-K 操作，因此 TOP-V 在 Ara RISC-V 向量处理器中增加可配置排序单元，并通过自定义 `VTOPK` 指令暴露。报告实现相对于基于 RVV 的排序最高达到 5.24× 加速。即使周边计算已经适合向量处理器，不规则且依赖密集的工作仍可能适合专用硬件。

集群内加速器采用互补路线。RedMulE 是 PULP RISC-V 集群中面向浮点 GEMM 和类 GEMM 操作的紧耦合矩阵引擎 [15]。它将专用引擎连接到共享集群存储器，并由可编程核协调运行。

在这些系统中，专用化位于功能单元、集群协处理器或向量处理器；SMU 接口暴露在线合并状态，以及交替标量状态的管理权。

SMU 发出自己的 TCDM 请求，因此标准 Spatz/RVV 执行不受影响。自定义接口将工作负载持久化配置（`OMCFG`）与反复执行（`OMERGE`）分开，且仅将交替标量状态选择位放入硬件，从而使专用化保持在规则向量数据通路之外，无需软件在每次合并时重复管理选择位。

C. AI 加速的软硬件协同设计

Gemmini 通过 RoCC 将可配置脉动阵列与 RISC-V 主机集成，并通过完整软件栈评估 [16]。Deeploy 将网络映射到异构 RISC-V 微控制器，在严格内存约束下为 RV32 核、机器学习扩展和 NPU 生成优化 C 代码 [17]。两者面向完整加速器或编译器映射；本文接口专门处理向量集群中持久化标量状态上的递推。对于在线注意力，软件保留分数构建、分块管理、首块初始化、输出缓冲区管理和 RVV，而 SMU 处理标量递推及其交替状态选择位。

FPGA 注意力加速器涵盖多 FPGA 文本生成、密集或编译器生成的数据通路、训练加速器以及稀疏性感知设计 [20], [21], [22], [23], [24]。Edge-MoE 将任务级稀疏性应用于多任务视觉 Transformer 加速器，TinyML 研究则评估异构部署流程 [25], [26]。其评估范围从加速器与算子测量延伸到部署；本文的问题是可编程向量集群应负责在线合并的哪一部分。

表 VII 引用的报告结果仅用于说明各工作的评估范围。

VII. 结论

本文提出面向 RISC-V 向量集群原生在线注意力的选择性软硬件映射：集群内标量在线合并单元（SMU）处理依赖状态的递推，Spatz/RVV 保留规则的 D 元素输出更新。阶段感知混合精度与 `OMCFG`/`OMERGE` 接口将非线性算术、持久化配置和交替标量状态选择位分开。

在匹配的 MMIO 控制原生工作负载中，该设计实现 8.524–17.784× 的递推加速；测量的 Native Core 收益较小，其窗口不包含 Q/K/V 加载和线性投影。相对于旧版标量设计，独立映射面积降低 27.613%，而布局前延迟代理指标上升。在两个显式 P 案例中，SMU 作为通用逐行 Softmax 替代没有一致收益，这支持对原生在线合并结果作数据流特定的解释。

参考文献

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[2] M. Milakov and N. Gimelshein, "Online normalizer calculation for Softmax," arXiv preprint arXiv:1805.02867, 2018, doi: 10.48550/arXiv.1805.02867.

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 16 344–16 359.

[4] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.

[5] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 68 658–68 685, doi: 10.52202/079017-2193. [Online]. Available: https://papers.nips.cc/paper_files/paper/2024/hash/7ede97c3e082c6df10a8d6103a2eebd2-Abstract-Conference.html

[6] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softermax: Hardware/software co-design of an efficient Softmax for transformers," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 469–474, doi: 10.1109/DAC18074.2021.9586134. [Online]. Available: https://doi.org/10.1109/DAC18074.2021.9586134

[7] A. Belano, Y. Tortorella, A. Garofalo, L. Benini, D. Rossi, and F. Conti, "A flexible template for edge generative AI with high-accuracy accelerated Softmax and GELU," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 15, no. 2, pp. 200–216, 2025.

[8] R. Wang, G. Islamoglu, A. Belano, V. Potocnik, F. Conti, A. Garofalo, and L. Benini, "VEXP: A low-cost RISC-V ISA extension for accelerated Softmax computation in transformers," in *Proc. IEEE Symp. Computer Arithmetic (ARITH)*, 2025, pp. 37–44, doi: 10.1109/ARITH64983.2025.00016.

[9] C. Peltekis, K. Alexandridis, and G. Dimitrakopoulos, "Reusing Softmax hardware unit for GELU computation in transformers," in *2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS)*, 2024, pp. 159–163, doi: 10.1109/AICAS59952.2024.10595882. [Online]. Available: https://doi.org/10.1109/AICAS59952.2024.10595882

[10] F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini, "Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads," *IEEE Transactions on Computers*, vol. 70, no. 11, pp. 1845–1860, 2021, doi: 10.1109/TC.2020.3027900. [Online]. Available: https://doi.org/10.1109/TC.2020.3027900

[11] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, "Ara: A 1-GHz+ scalable and energy-efficient RISC-V vector processor with multiprecision floating-point support in 22-nm FD-SOI," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 2, pp. 530–543, 2020, doi: 10.1109/TVLSI.2019.2950087. [Online]. Available: https://doi.org/10.1109/TVLSI.2019.2950087

[12] M. Perotti, S. Riedel, M. Cavalcante, and L. Benini, "Spatz: Clustering compact RISC-V-based vector units to maximize computing efficiency," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 7, pp. 2488–2502, 2025, doi: 10.1109/TCAD.2025.3528349.

[13] Q. Chen, C. Wang, Z. Wang, and J. Lin, "Top-V: A flexible and programmable Top-K acceleration framework based on the RISC-V ISA," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2026.

[14] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, 2019, document Version 20191213.

[15] Y. Tortorella, L. Bertaccini, L. Benini, D. Rossi, and F. Conti, "RedMulE: A mixed-precision matrix–matrix operation engine for flexible and energy-efficient on-chip linear algebra and TinyML training acceleration," *Future Gener. Comput. Syst.*, vol. 149, pp. 122–135, 2023.

[16] H. Genc *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. 58th ACM/IEEE Design Automation Conf. (DAC)*, 2021, pp. 769–774.

[17] M. Scherer *et al.*, "Deeploy: Enabling energy-efficient deployment of small language models on heterogeneous microcontrollers," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 11, pp. 4009–4020, 2024.

[18] Y. Li, W. Cao, X. Zhou, and L. Wang, "A low-cost reconfigurable nonlinear core for embedded DNN applications," in *2020 International Conference on Field-Programmable Technology (ICFPT)*, 2020, pp. 35–38, doi: 10.1109/ICFPT51103.2020.00014. [Online]. Available: https://doi.org/10.1109/ICFPT51103.2020.00014

[19] Q.-X. Wu, S.-S. Teng, M.-D. Shieh, C.-T. Huang, and J.-M. Lu, "A low-complexity and reconfigurable design for nonlinear function approximation in transformers," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 72, no. 12, pp. 1882–1886, 2025, doi: 10.1109/TCSII.2025.3585109. [Online]. Available: https://doi.org/10.1109/TCSII.2025.3585109

[20] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, "DFX: A low-latency multi-FPGA appliance for accelerating transformer-based text generation," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 616–630, doi: 10.1109/MICRO56248.2022.00051. [Online]. Available: https://doi.org/10.1109/MICRO56248.2022.00051

[21] E. Kabir, M. A. Kabir, A. R. J. Downey, J. D. Bakos, D. Andrews, and M. Huang, "FAMOUS: Flexible accelerator for the attention mechanism of transformer on UltraScale+ FPGAs," in *2024 International Conference on Field Programmable Technology (ICFPT)*, 2024, pp. 1–2, doi: 10.1109/ICFPT64416.2024.11113430. [Online]. Available: https://doi.org/10.1109/ICFPT64416.2024.11113430

[22] P. Plagwitz, F. Hannig, and J. Teich, "TRAC: Compilation-based design of transformer accelerators for FPGAs," in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*, 2022, pp. 17–23, doi: 10.1109/FPL57034.2022.00015. [Online]. Available: https://doi.org/10.1109/FPL57034.2022.00015

[23] H. Shao, J. Lu, M. Wang, and Z. Wang, "An efficient training accelerator for transformers with hardware-algorithm co-optimization," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 11, pp. 1788–1801, 2023, doi: 10.1109/TVLSI.2023.3305569. [Online]. Available: https://doi.org/10.1109/TVLSI.2023.3305569

[24] H. Wang, K. Wang, Y. Zhang, and J. Yu, "FAS-Trans: Fully exploiting FFN and attention sparsity for transformer on FPGA," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–9, doi: 10.1145/3676536.3676743. [Online]. Available: https://doi.org/10.1145/3676536.3676743

[25] R. Sarkar, H. Liang, Z. Fan, Z. Wang, and C. Hao, "Edge-MoE: Memory-efficient multi-task vision transformer architecture with task-level sparsity via mixture-of-experts," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 01–09, doi: 10.1109/ICCAD57390.2023.10323651. [Online]. Available: https://doi.org/10.1109/ICCAD57390.2023.10323651

[26] P. Wiese, G. İslamoğlu, M. Scherer, L. Macan, V. J.-B. Jung, A. Burrello, F. Conti, and L. Benini, "Toward attention-based TinyML: A heterogeneous accelerated architecture and automated deployment flow," *IEEE Design & Test*, vol. 42, no. 5, pp. 63–72, 2025, doi: 10.1109/MDAT.2025.3527371. [Online]. Available: https://doi.org/10.1109/MDAT.2025.3527371